

```

!-
!-Created 15/04/2021 By Roger Williams
!-
!-SubSunk-Rog Edition! - retail title: ROGSunk!
!-
!-uses machine code routine (waves.asm) to draw the sea can be in two
!-different waves (faking animation!)
!-setting mem loc 50000 to 0 or 1 for the different wave types
!-
!-current wave chars 78=/ and 77=\
!-
!-Program structure:
!-
!-0-90          init vars, clear screen
!-
!-100-195       keyboard input
!-200-499       draw sub and bomb if dropped check for collision
!-500-599       draw score and other player specific data
!-600-          sub sunk handler
!-
!-
!-
!-
!-8000-8899     intro screen
!-8900-14999    load machine code routines
!-9000-9199     asm for wave forward
!-9200-9300     asm for bomb draw
!-9400-9600     asm for draw/move boat
!-9700-9999     asm for draw/move sub
!-11000-11200   asm for sub hit
!-11300-11499   asm for boat sound
!-11500-11600   asm for clearing bottom 3 lines
!-11700-11899   asm for sub move sound
!-11900-12599   asm for intro text
!-12600-12799   asm for intro text colour change
!-12800-13100   asm for wave reversed
!-13200-14150   asm for "loading" progress bar (loaded first)
!-15000-        display "loading" text during asm load
!-
!-
!-VARS
!-
!-y= direction
!-sc=score
!-sh=lives (default 3)
!-bm=bomb dropped?
!-bh=bomb hit?
!-x1=bomb 1 x
!-x2=bomb 2 x
!-y1=bomb 1 y
!-y2=bomb 2 y
!-bc=bomb count (max 2)
!-m=user pressed movement key?
!-co=collisions happened?
!-cb=clear bottom row of screen (bomb missed)
!-sl=num of subs left

```

```

!-s1=num of subs to sink level 1 (s2 = level 2 etc)
!-cl=level text colour
!-
!-
!-
!-
!-Machine Code Routine Calls:
!-
!-sys 49152      - draw waves forward
!-sys 49320      - draw waves reversed
!-sys 49964      - draw bomb
!-sys 49730      - draw/move boat
!-sys 49900      - draw/move sub
!-sys 50100      - flash sub colours and flash/show BOOM! text (sub hit)
!-sys 50290      - boat engine sound
!-sys 50500      - clear bottom 3 lines
!-sys 50600      - sub move sound
!-sys 50800      - draw intro text
!-sys 51400      - change intro text colour
!-gosub 15000    - "loading" progress bar during asm load
!-
!-
!-mem loc used by assembler shifted 02/05/2021 to 53230
!-
near top of mem on purpose as exisitng asm programs no where near that area!
!-gives room for 17 ""variables"
!-
!-Draw waves:
!-
!-      53231      - first wave char  77= /
!-      53232      - second wave char 78= \
!-
!-Bomb draw
!-
!-      53233      - x pos
!-      53234      - y pos
!-
!-Boat draw/move
!-
!-      53235      - boat pos
!-
!-Sub draw/move
!-
!-      53236      - sub pos
!-      53237      - sub dir 0=r 1=l
!-
!-Sub hit
!-
!-      53238      - internally used for colour iteration
!-
!-boat sound
!-
!-      53239      - hi note value
!-      53240      - lo note value
!-  sound iterates between hi:4, lo:180 and hi:4, lo:12
!-

```

```

!-sub move sound
!-
!-
!-      53241 - hi note value
!-      53242 - lo note value
!- sound iterates between hi:8, lo:255 and hi:5, lo:71
!-
!-intro text colour
!-
!-      53243 - text colour
!-
!-Global:
!-
!-      53244 - level 1-3 (4=intro)
!-
!-
!-
!-
10 y=0:sc=0:sh=3:bm=0:bh=0:m=0:x1=0:x2=0:y1=0:y2=0:bc=0:co=0:cb=0:sl=3:s1=3
15 cl=0:poke 53245,10:poke 53246,10:zz=1:rem for progress bar
19 rem set mem locs default values for asm funcs
20 poke 53233,5:poke 53234,y:rem set base value for bomb x/y pos
30 poke 53231,78:poke 53232,77:rem set wave chars and type
38 poke 53235,0:rem set boat default pos
40 poke 53244,1:rem set to level 1
45 poke 53236,0:poke 53237,0:rem sub pos and dir
49 rem show loading progress bar and load machine code routines
50 gosub 8910: print chr$(147)
59 rem show intro here
60 gosub 8000
90 print chr$(147):poke 198,0:sys 49730:rem draw boat first time
94 sys 49900:rem draw sub first time
99 rem get keyboard input
100 get a$:rem poke 53239,180:poke53240,4:sys 50290:rem boat engine sound
110 if a$="q" then end
114 gosub 500:rem show score etc
115 rem poke 53239,255:poke53240,8:sys 50600:rem sub move sound
119 rem draw waves forward
120 sys 49152:rem wave type 1
130 if a$="a" then y=y-1:m=1
140 if a$="d" then y=y+1:m=1
150 if a$="z" then bm=1:bc=bc+1
152 if bc=1 then if y1=0 then y1=y+2:x1=5
154 if bc=2 then if y2=0 then y2=y+2:x2=5
156 if bc >2 then bc=2
160 if y<0 then y=0
170 if y>40 then y=40
180 if m=1 then poke 53235,y:sys 49730:rem if movement of boat then draw it!
189 rem draw waves reversed
190 sys 49320:rem wave type 2
197 gosub 210
198 goto 100
200 rem bomb drop and collision detection
201 rem bomb (if dropped) max 2 bombs each has same x pos but diff y pos
210 if bm=1 then if bc=1 then x1=x1+1
220 if bm=1 then if bc=2 then x2=x2+1

```

```

230 if bm=1 then if x1>24 then bc=0:x1=0:cb=1:rem cancel bomb bottom of scr
240 if bm=1 then if x2>24 then bc=bc-
1:x2=0:cb=1:rem cancel bomb bottom of scr
249 rem check for collision before drawing the bomb
250 if x1 =23 then if peek(1944+y1) <> 32 then if peek
(1944+y1) <> 81 then co=1
260 if x2 =23 then if peek(1944+y2) <> 32 then if peek
(1944+y1) <> 81 then co=1
270 if co=1 then 280:rem if collision don't bother drawing the bomb!
275 rem draw bomb
276 if x1 <> 0 then :poke 53233,x1:poke 53234,y1:sys 49664
278 if x2 <> 0 then :poke 53233,x2:poke 53234,y2:sys 49664
279 rem if collision decrement bomb count
280 if co <> 0 then bc=bc-1:goto 600:rem sub sunk!
289 rem if bomb reached bottom of screen clear bottom line
290 if cb = 1 then for a = 1984 to 2023:poke a,32:next a
300 if bc=1 then x2=0
310 if bc<=0 then x2=0:x1=0:bm=0:bc=0:y1=0:y2=0
319 rem move sub
320 if co=0 then if peek(53237) =0 then dr=2
330 if co=0 then if peek(53237) =1 then dr=1
332 if dr=1 then if peek(53236) =0 then poke 53236,0:poke 53237,0
334 if dr=1 then if peek(53236) <>0 then poke 53236,(peek(53236)-1)
336 if dr=2 then if peek(53236) =34 then poke 53236,34:poke 53237,1
337 if dr=2 then if peek(53236) <>34 then poke 53236,(peek(53236)+1)
360 sys 49900:rem draw sub
379 rem reset collision var and clear bottom row var
380 co=0:cb=0:dr=0
400 rem poke 53239,12:poke53240,4:sys 50290:rem boat engine sound
499 return
500 rem draw score and other player specific data
505 poke781,..:poke782,..:poke783,..:sys65520
510 printchr$(18);chr$(5);"score: ";sc;tab(32);"Subs: ";sl
520 poke 1044,12:poke1045,5:poke1046,22:poke1047,5:poke1048,12:poke1049,58
530 poke 1051,peek(53244)+48:rem convert level number into petSCII char
540 poke55316,cl:poke55317,cl:poke55318,cl:poke55319,cl::poke55320,cl
550 poke55321,cl::poke55322,cl::poke55323,cl
560 cl=cl+1
570 if cl>16 then cl=0
580 poke 53239,5:poke53240,71:sys 50600:rem sub move sound
598 return
599 rem sub sunk!
600 if peek(53244)=1 then sl=sl-1:sc=sc+100:cl=0
610 if sl=0 then sys 50100:goto 7000:rem game over player won!
620 if sl > 0 then poke 781,10:poke 782,15:poke783,..:sys65520:print " "
630 if sl > 0 then 100:rem loop
640 sys 50100:rem flash sub and show/flash boom! text
6999 rem gameover!
7000 input "play again? y/n: ";a$
7010 if a$="n" then end
7020 if peek(53244)<> 3 then poke 53244,peek(53244)+1
7030 if peek(53244)= 3 then poke 53244,1:print "no more levels! -
press a key"
7040 if peek(53244)= 3 then poke 198,0:wait 198:poke 198,0
7090 goto 90:rem restart if not no
7999 rem intro screen

```

```

8000 poke 53244,4:a=0:ic=0:rem press a key text colour
8005 sys 50800:rem draw intro text
8010 sys 49152:rem wave type 1
8020 if peek(198) <> 0 then 90:rem wait for key press
8030 poke 53243,a:a=a+1:if a>16 then a=0
8040 sys 51400:rem change text colour
8060 poke 781,22:poke782,15:poke783,..:sys65520:print"press a key"
8069 rem flash text colour
8070 poke 56191,ic:ic=ic+1:poke 56191,ic:ic=ic+1:poke 56192,ic:ic=ic+1
8080 poke 56193,ic:ic=ic+1:poke 56194,ic:ic=ic+1:poke 56195,ic:ic=ic+1
8090 poke 56196,ic:ic=ic+1:poke 56197,ic:ic=ic+1:poke 56198,ic:ic=ic+1
8100 poke 56199,ic:ic=ic+1:poke 56200,ic:ic=ic+1:poke 56201,ic:ic=ic+1
8110 if ic>14 then ic=0
8120 rem draw waves reversed
8130 sys 49320:rem wave type 2
8190 goto 8020
8200 poke 198,0:rem delete pressed key
8290 poke 53244,1:rem set game to level 1
8300 end
8399 return
8900 rem asm for waves forward
8910 sa = 49152:zz=1:yt=10:yu=10:rem set loading asm text
8920 for n = 0 to 166
8930 read a : poke sa+n,a:gosub 15000: next n
8940 data 162,0,172,252,207,192,0,208,2,96,0,192,2,240,43,192
8960 data 3,240,74,192,4,240,105,169,14,157,168,218,173,239,207,157
8980 data 168,6,232,224,40,240,121,169,14,157,168,218,173,240,207,157
9000 data 168,6,232,224,40,208,212,76,160,192,169,14,157,224,217,173
9020 data 239,207,157,224,5,232,224,40,240,86,169,14,157,224,217,173
9040 data 240,207,157,224,5,232,224,40,208,224,76,160,192,169,14,157
9060 data 160,216,173,239,207,157,160,4,232,224,40,240,51,169,14,157
9080 data 160,216,173,240,207,157,160,4,232,224,40,208,224,76,160,192
9100 data 169,14,157,192,219,173,239,207,157,192,7,232,224,40,240,16
9120 data 169,14,157,192,219,173,240,207,157,192,7,232,224,40,208,224
9140 data 169,14,141,134,2,96,0
9200 rem asm for depth charge
9210 sa = 49664:zz=2:rem set loading asm text
9220 for n = 0 to 51
9230 read a% : poke sa+n,a%: gosub 15000:next n
9240 data 173,241,207,201,4,240,24,169,0,174,241,207,202,172,242,207
9260 data 24,32,240,255,169,6,141,134,2,169,113,32,210,255,232,169
9280 data 0,172,242,207,24,32,240,255,169,0,141,134,2,169,113,32,210,255,96,0
9410 rem asm for boat
9420 sa = 49730:zz=3:rem set loading asm text
9430 for n = 0 to 89
9440 read a% : poke sa+n,a%: gosub 15000: next n
9450 data 169,0,162,0,160,0,169,32,157,40,4,157,80,4,157,120
9470 data 4,232,224,40,208,242,172,243,207,169,118,153,42,4,169,160
9490 data 153,82,4,169,95,153,120,4,169,160,153,121,4,169,160,153
9510 data 122,4,169,160,153,123,4,169,105,153,124,4,169,1,153,42
9530 data 216,153,82,216,153,120,216,153,121,216,153,122,216,153,123,216
9550 data 153,124,216,169,14,141,134,2,96,0
9700 rem asm for sub draw
9710 sa = 49900:zz=4:rem set loading asm text
9720 for n = 0 to 171
9730 read a% : poke sa+n,a%:gosub 15000: next n

```

```

9740 data 169,0,162,0,160,0,169,32,157,112,7,157,152,7,157,192
9760 data 7,232,224,40,208,242,172,244,207,173,245,207,201,0,240,13
9780 data 169,108,153,115,7,169,123,153,116,7,76,35,195,169,108,153
9800 data 113,7,169,123,153,114,7,169,108,153,152,7,169,160,153,153
9820 data 7,169,160,153,154,7,169,160,153,155,7,169,160,153,156,7
9840 data 169,123,153,157,7,169,124,153,192,7,169,160,153,193,7,169
9860 data 160,153,194,7,169,160,153,195,7,169,160,153,196,7,169,126
9880 data 153,197,7,169,5,153,113,219,153,114,219,153,115,219,153,116
9900 data 219,153,152,219,153,153,219,153,154,219,153,155,219,153,156,219
9920 data 153,157,219,153,192,219,153,193,219,153,194,219,153,195,219,153
9940 data 196,219,153,197,219,169,14,141,134,2,96,0
11000 rem asm for sub hit
11010 sa = 50100:zz=5:rem set loading asm text
11020 for n = 0 to 89
11030 read a% : poke sa+n,a%: gosub 15000:next n
11040 data 169,0,162,0,160,0,169,2, 141,160,5,169,15,141,161,5
11060 data 169,15,141,162,5,169,13,141,163,5,169,33,141,164,5,169
11080 data 0,141,246,207,153,112,219,153,152,219,153,192,219,153,144,217
11100 data 200,192,40,240,3,76,216,195,238,246,207,160,0,162,0,232
11120 data 224,255,208,251,173,246,207,201,255,240,3,76,216,195,162,0
11140 data 141,246,207,169,14,141,134,2,96,0
11300 rem asm for boat move sound
11310 sa = 50290:zz=6:rem set loading asm text
11320 for n = 0 to 80
11330 read a% : poke sa+n,a%:gosub 15000: next n
11340 data 169,0,162,0,169,0,173,247,207,201,0,240,66,169,15,141
11360 data 24,212,141,0,212,141,1,212,141,2,212,141,4,212,169,128
11380 data 141,4,212,169,0,169,15,141,24,212,169,5,141,5,212,169
11400 data 0,141,10,212,169,15,141,6,212,169,15,141,12,212,173,249
11420 data 207,141,1,212,173,250,207,141,0,212,169,129,141,4,212,96,0
11500 rem asm for sub clear
11510 sa = 50500:zz=7:rem set loading asm text
11520 for n = 0 to 29
11530 read a% : poke sa+n,a%: gosub 15000:next n
11540 data 169,32,162,0,157,112,7,157,152,7,157,192,7,232,224,40
11560 data 208,242,162,0,141,246,207,169,14,141,134,2,96,0
11700 rem asm for sub move sound
11710 sa = 50600:zz=8:rem set loading asm text
11720 for n = 0 to 75
11730 read a% : poke sa+n,a%: gosub 15000:next n
11740 data 169,0,162,0,169,0,173,249,207,201,0,240,61,141,7,212
11760 data 141,8,212,141,12,212,141,13,212,169,128,141,11,212,169,0
11780 data 169,15,141,24,212,169,5,141,12,212,169,0,141,17,212,169
11800 data 15,141,13,212,169,15,141,19,212,173,249,207,141,8,212,173
11820 data 250,207,141,7,212,169,129,141,11,212,96,0
11900 rem asm for intro text
11910 sa = 50800:zz=9:rem set loading asm text
11920 for n = 0 to 531
11930 read a% : poke sa+n,a%:gosub 15000: next n
11940 data 169,0,162,0,169,6,141,32,208,169,160,141,5,4,141,6
11960 data 4,141,7,4,141,8,4,141,9,4,141,10,4,141,12,4
11980 data 141,13,4,141,14,4,141,15,4,141,16,4,141,17,4,141
12000 data 19,4,141,20,4,141,21,4,141,22,4,141,23,4,141,24
12020 data 4,141,45,4,141,50,4,141,52,4,141,57,4,141,59,4
12040 data 141,85,4,141,90,4,141,92,4,141,97,4,141,99,4,141
12060 data 125,4,141,130,4,141,132,4,141,137,4,141,139,4,141,165

```

```

12080 data 4,141,166,4,141,167,4,141,168,4,141,169,4,141,170,4
12100 data 141,172,4,141,177,4,141,179,4,141,205,4,141,206,4,141
12120 data 212,4,141,217,4,141,219,4,141,245,4,141,247,4,141,252
12140 data 4,141,1,5,141,3,5,141,5,5,141,6,5,141,7,5
12160 data 141,8,5,141,29,5,141,32,5,141,36,5,141,41,5,141
12180 data 43,5,141,46,5,141,69,5,141,73,5,141,76,5,141,77
12200 data 5,141,78,5,141,79,5,141,80,5,141,81,5,141,83,5
12220 data 141,84,5,141,85,5,141,86,5,141,152,5,141,153,5,141
12240 data 154,5,141,155,5,141,156,5,141,157,5,141,159,5,141,164
12260 data 5,141,166,5,141,171,5,141,173,5,141,192,5,141,199,5
12280 data 141,204,5,141,206,5,141,207,5,141,211,5,141,213,5,141
12300 data 218,5,141,232,5,141,239,5,141,244,5,141,246,5,141,248
12320 data 5,141,251,5,141,246,5,141,253,5,141,1,6,141,16,6
12340 data 141,23,6,141,28,6,141,30,6,141,33,6,141,35,6,141
12360 data 37,6,141,40,6,141,56,6,141,63,6,141,68,6,141,70
12380 data 6,141,74,6,141,75,6,141,77,6,141,79,6,141,96,6
12400 data 141,97,6,141,98,6,141,99,6,141,100,6,141,101,6,141
12420 data 103,6,141,108,6,141,110,6,141,115,6,141,117,6,141,118
12440 data 6,141,141,6,141,143,6,141,148,6,141,150,6,141,155,6
12460 data 141,157,6,141,159,6,141,181,6,141,183,6,141,188,6,141
12480 data 190,6,141,195,6,141,197,6,141,200,6,141,221,6,141,223
12500 data 6,141,228,6,141,230,6,141,235,6,141,237,6,141,241,6
12520 data 141,5,7,141,7,7,141,12,7,141,14,7,141,19,7,141
12540 data 21,7,141,26,7,141,40,7,141,41,7,141,42,7,141,43
12560 data 7,141,44,7,141,45,7,141,47,7,141,48,7,141,49,7
12580 data 141,50,7,141,51,7,141,52,7,141,54,7,141,59,7,141,61,7,96,0
12600 rem asm for intro text colour
12610 sa = 51400:zz=zz=10:rem set loading asm text
12620 for n = 0 to 94
12630 read a% : poke sa+n,a%:gosub 15000: next n
12640 data 169,0,162,0,173,251,207,157,0,216,232,224,240,240,3,76
12660 data 207,200,162,0,157,240,216,232,224,120,240,3,76,220,200,200
12680 data 192,255,240,5,76,231,200,160,0,200,192,255,240,5,76,241
12700 data 200,160,0,200,192,255,240,3,76,251,200,160,0,162,0,238
12720 data 251,207,173,251,207,157,144,217,232,224,240,240,3,76,13,201
12740 data 162,0,157,88,218,232,224,240,240,3,76,26,201,96,0
12800 rem asm for waves reversed
12810 sa = 49320:zz=11:rem set loading asm text
12820 for n = 0 to 164
12830 read a% : poke sa+n,a%:gosub 15000: next n
12840 data 172,252,207,192,0,208,2,96,0,192,2,240,43,192,3,240
12860 data 74,192,4,240,105,169,14,157,168,218,173,240,207,157,168,6
12880 data 232,224,40,240,121,169,14,157,168,218,173,239,207,157,168,6
12900 data 232,224,40,208,212,76,70,193,169,14,157,224,217,173,240,207
12920 data 157,224,5,232,224,40,240,86,169,14,157,224,217,173,239,207
12940 data 157,224,5,232,224,40,208,224,76,70,193,169,14,157,160,216
12960 data 173,240,207,157,160,4,232,224,40,240,51,169,14,157,160,216
12980 data 173,239,207,157,160,4,232,224,40,208,224,76,70,193,169,14
13000 data 157,192,219,173,240,207,157,192,7,232,224,40,240,16,169,14
13020 data 157,192,219,173,239,207,157,192,7,232,224,40,208,224,169,14
13040 data 141,134,2,96,0
14998 return
14999 rem text shown while loading asm routines
15000 if zz <>1 then 15060:rem skip drawing box etc if >1
15005 poke 781,4:poke782,12:poke783,.:sys 65520
15010 print chr$(5);chr$(19);"loading"

```

```

15020 rem draw box to contain progress bar
15030 poke 1433,116:poke 1455,106
15040 for a=0 to 22:poke 1393+a,111:poke1473+a,119:next a
15059 rem show what type of asm data being loaded into memory
15060 if zz=1 then tx$=" wave forward assembler data ":poke53280,zz
15070 if zz=2 then tx$=" depth charge assembler data ":poke53280,zz
15080 if zz=3 then tx$=" boat draw assembler data ":poke53280,zz
15090 if zz=4 then tx$=" sub draw assembler data ":poke53280,zz
15100 if zz=5 then tx$=" sub hit assembler data ":poke53280,zz
15110 if zz=6 then tx$="boat engine sound assembler data":poke53280,zz
15120 if zz=7 then tx$=" clear sub assembler data ":poke53280,zz
15130 if zz=8 then tx$=" sub engine sound assembler data":poke53280,zz
15140 if zz=9 then tx$=" intro text assembler data ":poke53280,zz
15150 if zz=10 then tx$=" intro text flash assembler data":poke53280,zz
15160 if zz=11 then tx$=" wave reversed assembler data ":poke53280,zz
15170 if zz <> 0 then poke 781,8:poke782,6:poke783,.:sys 65520
15180 if zz <> 0 then print tx$:zz=0
15190 poke 1424+yt,160:poke 55696+yt,1:yt=yt+1
15200 if yt>30 then fora=0to20:poke55706+tu+a,6:nexta:yt=yt+1
16000 return

```